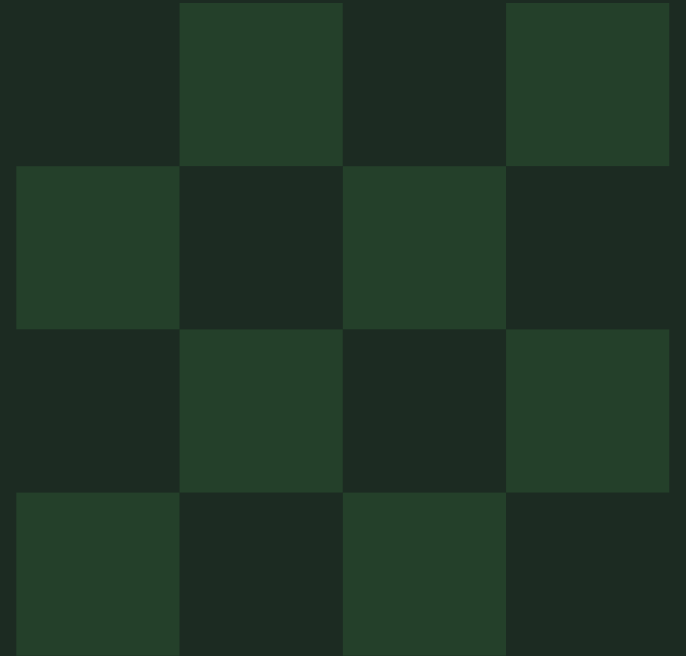


A Bitboard Chess Engine with Alpha-Beta Search

Move generation verified by perft · transposition table · time-budgeted levels

Faris Muratovic · 19958

Artificial Intelligence · Instructor: Prof. Dr. Izudin Dzafic



The board, and proving the rules are right

Twelve 64-bit integers — one bit per square, one board per piece type

```
uint64_t  whitePawns, whiteKnights, ...
```

“Which pieces attack this square?” becomes a handful of AND / OR / shift operations and a population count — the reason every strong engine uses bitboards.

Perft: count every legal leaf, compare with published values

Start position, depth 5	4 865 609	✓ match
Kiwipete, depth 4	4 085 603	✓ match
Position 3, depth 5	674 624	✓ match
Position 4, depth 4	422 333	✓ match



Castling, en passant, promotion and pins — all exercised by the four test positions

How the opponent thinks

Negamax with alpha-beta: prove a move is too good for the opponent to allow, and stop searching it

Move ordering

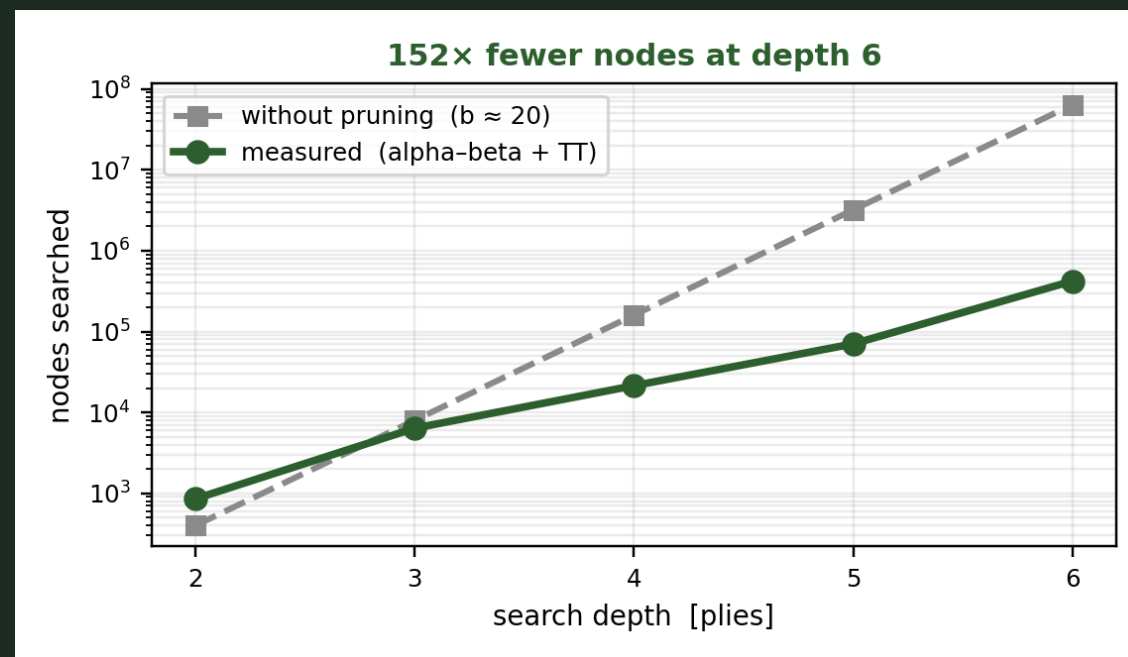
captures by MVV-LVA, killer and history heuristics — cutoffs come early only if good moves go first

Transposition table

Zobrist hash, 16 MB;
the same position reached by another move order is not re-searched

Quiescence

keep searching captures past the horizon so the engine does not hang a piece one ply too late



Measured effective branching factor: 2 – 6, against 20 – 48 legal moves per position

Difficulty is a clock, not a depth

One extra ply costs 10× more in a dense middlegame than in a sparse endgame

Easy

150 ms

Normal

400 ms

Hard

1000 ms

Expert

2200 ms

per move — the search starts another iteration only if it estimates it will finish in time



Iterative deepening

depth 1, 2, 3 ... — each pass makes the next one faster by ordering the best move first



Root-parallel final pass

root moves spread over all cores, each thread with its own board and table

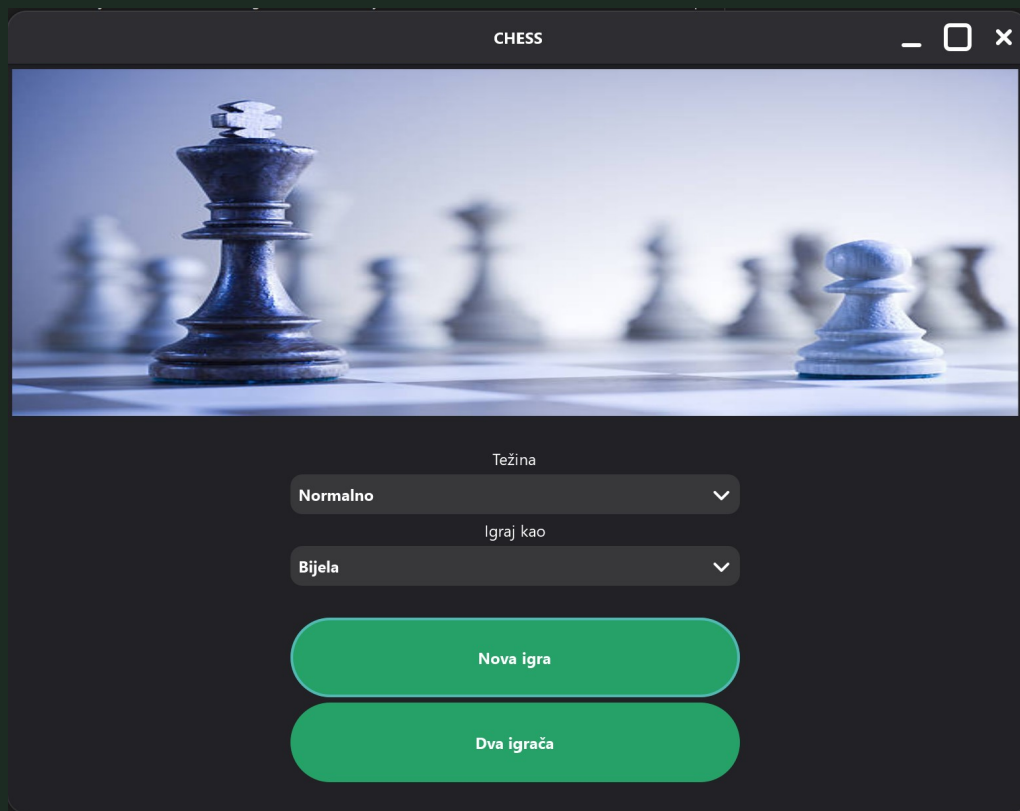


Worker thread

the search never runs on the UI thread, so the board keeps painting while the bot thinks

The application

natID / natGUI · drag-and-drop board drawn on a canvas



- Human vs human, or vs the bot on either colour
- Legal destinations highlighted on pick-up
- Promotion picker, undo and redo
- Captured-piece panels, Bosnian / English UI
- Draws: stalemate, 50-move, repetition, material

The engine compiles stand-alone — every number in the report was measured without the GUI.

What I take from it

1

Make errors visible

Perft and an incrementally maintained hash caught two move-generation bugs that no amount of play-testing would have isolated.

2

Measure before choosing

Difficulty levels, time budgets and the per-ply growth estimate all came from measurement, not from guessing what felt right.

3

Keep the search off the UI

Copy the position, release the lock, search on the copy — the first version froze the whole interface for the bot's entire think time.

Bitboards + alpha-beta + a transposition table: a complete, verified opponent in 2 600 lines of engine code.